

RoboCall AI

What it is, how it is built, and where it stands

24 September 2026 · Phase 1: **complete** · Carrier: **Telnyx, funded and live** · Automated checks: **283 passing**

Summary. Phase 1 is finished and proven end to end. The application dials through **Telnyx**, and on 24 September a real handset rang and heard the script read aloud in a neural voice. The migration off Twilio is complete: no Twilio code, credentials or dependency remains in the running system. **283 automated checks pass** against a fake carrier, and the first live call exposed a genuine defect that the test suite could not have caught on its own — now fixed and covered. Total spend to date is **\$1.24**, against a balance of \$8.76.

1 What the project is

RoboCall AI is a web dashboard that places outbound phone calls and has a text-to-speech voice read a prepared script aloud. It is built for announcement campaigns — the canonical case being a shop telling its customer list that a sale starts Friday.

You write a message, or have Claude draft one from a one-line brief. You paste or upload a list of numbers. You choose a schedule. From there a background engine does the work: it paces the calls so a burst does not get the number flagged as spam, holds each call until it is inside legal calling hours *in the recipient's own timezone*, screens every number against a do-not-call list, and writes each outcome back to a call log with the carrier's own call ID.

Phase 1 is play-and-read. The system dials, identifies the caller, speaks, offers an opt-out and hangs up. It does not listen and it does not converse. That is Phase 3, and it is a different architecture — a live audio stream rather than a document handed to the carrier.

2 What it is built from

Layer	Choice	Why this one
Carrier	Telnyx, via its TeXML API	Roughly half Twilio's per-minute cost. TeXML takes the whole call script inline on the dial request, so a laptop can place a working call with nothing exposed to the internet.
Backend	Python 3, FastAPI, uvicorn	One process serves the JSON API, the dashboard and the carrier webhooks. The dispatcher runs inside it as a daemon thread, so campaigns continue whether or not a browser is open.
HTTP	httpx, direct against the REST API	The surface we need is four endpoints wide. Telnyx's errors are more useful raw than wrapped in a vendor SDK, and there is no SDK version to track.
Database	SQLite, via the standard library	One file, no server. Kept outside the project folder by default — Dropbox replaces synced files underneath a running app and will destroy an open database.
Frontend	Hand-written HTML, CSS and JavaScript	No build step, no framework, no node_modules. Edit a file, reload the page.

Layer	Choice	Why this one
Voice	AWS Polly neural voices through Telnyx	Telnyx's own engines are available and cheaper; Polly is the default because it does not sound like a 2005 IVR.
Drafting	Claude, optional	Turns a one-line brief into a compliant script. Everything else works without it.
Reports	ReportLab	This document is generated from code and can be rebuilt.

The dependency list is deliberately short: FastAPI, uvicorn, httpx, pydantic and python-dotenv, plus *anthropic* and *reportlab* for the two optional pieces. A campaign firing does not place N calls — it writes N rows to a task table that a background loop drains. That separation is what makes pacing, quiet hours and late opt-outs possible at all.

3 The move from Twilio to Telnyx

The system was originally written against Twilio and never placed a successful call there: a Twilio trial account rejects the inline TwiML that direct mode depends on, so every attempt failed at the carrier. Rather than pay to unblock a more expensive provider, the integration was rewritten for Telnyx.

Telnyx offers two voice APIs, and the choice between them decided the shape of the app:

Option	Verdict
Call Control	Rejected. It is event-driven — Telnyx tells you a call was answered over a webhook and waits for you to send a <i>speak</i> command back. That makes a publicly reachable server mandatory just to place one working call.
TeXML	Chosen. Its call-creation endpoint accepts the entire XML document inline as the <i>Texml</i> parameter, so the script travels with the dial request. This is what keeps the laptop-only setup the rest of the app is built around.

What changed, concretely:

- **telephony.py** rewritten against the Telnyx REST API using httpx. The *twilio* package is gone from requirements.txt entirely.
- Configuration keys moved to *telnyx_**; webhook routes moved to */telnyx/**.
- Webhook authentication changed from Twilio's HMAC scheme to Telnyx's **Ed25519** signatures, with replay protection on the timestamp.
- Error translation rewritten around the failures that actually strand a new Telnyx account: an empty balance, a missing outbound voice profile, a wrong application ID, and a From number the account does not own.
- Voice identifiers that only existed on Twilio are now replaced rather than passed through, so an old saved campaign cannot dial with a voice Telnyx will reject.

One capability did not survive the move. Twilio offered **Verified Caller ID**, which let you present a number you owned elsewhere. Telnyx has no equivalent: the From number must be one you own on the account. In practice this is not a loss — presenting another carrier's number earns lower STIR/SHAKEN attestation, which makes handsets *more* likely to label the call *Spam Likely*.

4 Proof that it works

Two live calls were placed to a real mobile on 24 September. Both are recorded in the dashboard's call log with the Telnyx call ID returned by the carrier. Call SIDs are abbreviated below.

When	Outcome	Telnyx SID	What actually happened
15:09 UTC	Delivered, voicemail — 30s	v3:mL8Q...1bsw	Connected, but the recipient heard nothing . Answering-machine detection misclassified a live answer and held the script for its full window.
15:12 UTC	Delivered — 21s	v3:-AON...uSmQ	Connected and spoke . The script was read aloud in a neural voice and the recipient confirmed hearing it.

The defect the first call found

The first call is the more valuable of the two. It connected, it was billed, the carrier reported success — and the person holding the phone heard thirty seconds of silence. Nothing in a test suite built on a fake carrier could have caught it, because the fake faithfully returned exactly what the real API returns.

The cause was two defects stacked on each other:

- **The wrong default for a test call.** A one-off call from the *Call me now* button has no campaign attached, so the answering-machine setting was null and fell through to *voicemail* mode — the mode whose entire purpose is to wait for a greeting to finish. But a person is holding the phone. There is nothing to wait for.
- **A detection window far longer than human patience.** Telnyx's own specification is explicit that *"execution is blocked until Answering Machine Detection is completed"*. The window was set to 30 seconds, so a live "hello" misread as a machine meant half a minute of dead air before the script would have started — long after any real recipient has hung up. This was not a test-call problem; it applied to every campaign call too.

Test calls now dial with detection disabled, campaign calls keep the mode they were configured with, and the detection window is cut to 12 seconds. That last number is a real trade-off rather than a free win: the cost of guessing short is talking over the tail of an unusually long voicemail greeting, which is plainly better than never being heard. Six new assertions pin all of it down.

5 Scheduling: frequency, windows, and what they cost

A campaign fires on a frequency — **once**, **hourly**, **daily** or **weekly** — at a chosen time of day, inside an optional window with a **begins** and an **ends** timestamp. Both are read in the campaign's own timezone, the same one the time of day uses, and stored in UTC.

The end timestamp is the one that matters. Without it a recurring campaign has no stopping condition: it keeps dialling and keeps billing until somebody remembers to pause it. The table below is the evidence that it does stop — and the last row is what happens when it is left blank.

Verified by simulation

Each scenario is driven through the **real dispatcher** against a scratch database and a stand-in carrier, on a simulated clock. The counts are the counts the live engine would produce, because it is the live engine producing them. Verifying a weekly campaign against a real clock would take two months, and an hourly one would bill for

every tick.

Scenario	Runs	Calls	Cost	Stops by itself
Hourly, bounded to 6 hours	6	18	\$0.18	yes
Hourly, bounded to 24 hours	24	24	\$0.24	yes
Daily, bounded to 7 days	7	70	\$0.70	yes
Daily, bounded to 30 days	30	1500	\$15.00	yes
Weekly, bounded to 8 weeks	8	200	\$2.00	yes
Once	1	10	\$0.10	yes
Daily, NO end date (30 days shown)	34	1700	\$17.00	no — runs forever

*Reproduce with `python docs/frequency_check.py`. Priced at **\$0.01 per call** — measured, not quoted: the account balance moved from \$8.77 to \$8.76 across a real 21-second delivered call.*

Read the last two rows together. The same daily campaign to the same 50 people costs **\$15** bounded to 30 days, and has *no upper bound at all* without an end date — it simply keeps going. That is the entire argument for the field. The \$2.00/day cap on the Telnyx profile is the backstop, but a cap stops a campaign by running out of money, which is a worse way to find out.

Nineteen further checks in the test suite cover the boundaries the table cannot show: an end date moved into the past retiring a campaign on the spot, a start date deferring the first run, an end that falls before the first slot being refused at creation rather than producing a campaign that can never fire, and a window entered as 10:00 New York being stored as 15:00 UTC rather than shifted by the operator's own offset.

6 How much of this is verified

Two plain Python scripts, no test framework. They run against a fake Telnyx in about a second, need no credentials and cost nothing, so there is no reason not to run them.

Suite	Checks	What it covers
smoke_test.py	75	The happy path: phone parsing, timezone resolution from area code, calling windows, schedule arithmetic across a daylight-saving change, consent and do-not-call screening, pacing, and the HTTP API end to end.
pressure_test.py	208	Deliberate abuse: eight threads racing the same queue, the exact parameter names sent to Telnyx, every failure that strands a first-run account, forged and replayed Ed25519 webhook signatures, a provider that throws mid-dial, schema drift from an older database, malformed numbers and times, TeXML injection, every API rejection path, directory-traversal attempts against the document viewer, and the campaign start/end window.
Total	283	All passing.

The suites assert against Telnyx's published parameter names, so a typo that would only surface as a failed live call is caught on a laptop instead. What they cannot check is whether audio actually reaches a human ear — which is exactly the gap the first live call filled.

7 Cost position and spending controls

Item	Amount	Note
Phone number	\$1.00 once, then \$1.00/month	+1 979-921-8143, a College Station number matching the account's service address.
Live test calls	about \$0.01 each	The balance moved from \$8.77 to \$8.76 across a 21-second call.
Balance remaining	\$8.76	Telnyx refuses to dial at zero.

Guard rails configured on the account

- **\$2.00 daily spend limit** on the outbound voice profile. This is the control that actually matters: a runaway loop stops at \$2, not at the whole balance.
- **US-only destinations**, so a malformed number cannot dial an international premium rate.
- **\$0.05/minute ceiling** — any route more expensive than that is refused.
- **Two concurrent channels**, which caps how fast a mistake can compound.
- In the application itself: a 12 calls/minute pace, a 09:00–20:00 window in each recipient's local time (tighter than the federal 08:00–21:00 rule), and do-not-call screening before every dial.

8 What is deliberately not done yet

Direct mode is what makes the app runnable on a laptop, and it is genuinely limited. Setting a public base URL switches the whole system to webhook mode and unlocks the rest; the code for both paths is already written and tested.

Not available in direct mode	Why
Press-9-to-opt-out during a call	The keypress has to be delivered to this server, which means the server must be reachable. The spoken opt-out instruction — call this number back — works in both modes.
Hanging up on voicemail (<i>live answers only</i>)	Requires the carrier to ask this server what to do at the moment it detects a machine.
Live status callbacks	Outcomes are polled from the Telnyx API instead, which is slower but correct.

Beyond that, **Phase 3 — a call that listens and answers back** — is not started and is not a small extension of this. It needs a bidirectional audio stream, speech recognition, and a model in the loop at conversational latency. Everything in this document is the play-and-read system underneath it.

9 Sensible next steps

- **Run a small real campaign** — five or ten consenting numbers — to exercise pacing, the calling window and the call log against real carrier behaviour rather than a fake.
- **Listen to a voicemail the system leaves**, now that the detection window has changed. The 12-second setting is a judgement call and one recorded greeting will confirm or correct it.
- **Try webhook mode once** behind a tunnel, to confirm press-9 opt-out end to end.
- **Decide the caller-identity story** before any volume: a single number making repeated calls gets labelled, and that is a reputation problem rather than a code problem.

Where this leaves things. The engine is built, tested and no longer theoretical — a phone rang and a voice read the script. What remains is operational judgement rather than construction: how fast to dial, from what number, and to whom.